ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №3

Студент Кольчугина Е. А.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

САМАРА 2025

Содержание

Задание 1 3

Задание 2 4

Задание 3 5

Задание 4 8

Задание 5 10

Задание 6 15

Задание 7 16

Задание 1

Изучены классы исключений:

- Exception - базовый класс для исключений.
  В Java выделяют проверяемые (checked) и непроверяемые (unchecked) исключения: Проверяемые — проверяются компилятором во время компиляции. Программист обязан обработать такие исключения, иначе программа не скомпилируется. Непроверяемые — не проверяются компилятором, возникают во время выполнения программы и обычно являются результатом ошибок программиста.
- IndexOutOfBoundsException - выход за границы индексов (при попытке обращения к элементу по индексу, выходящему за допустимые границы)
- ArrayIndexOutOfBoundsException - выход за границы массива (при попытке обращения к элементу массива по индексу, выходящему за размер массива)
- IllegalArgumentException - недопустимый аргумент
- IllegalStateException - недопустимое состояние объекта

Задание 2

C: > Users > lisako > Documents > oop > Lab-3-2025 > functions > FunctionPointIndexOutOfBoundsException.java > ...

```java
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException{

    public FunctionPointIndexOutOfBoundsException() {
        super(s: "Выход за границы набора точек функции");
    }

    public FunctionPointIndexOutOfBoundsException(String message) {
        super(message);
    }
}
```

C: > Users > lisako > Documents > oop > Lab-3-2025 > functions > InappropriateFunctionPointException.java > ...

```java
package functions;

public class InappropriateFunctionPointException extends Exception{

    public InappropriateFunctionPointException(){
        super(message: "Некорректная операция над точкой");
    }
    public InappropriateFunctionPointException(String message) {
        super(message);
    }
}
```

Были реализованы два класса исключений для работы с табулированными функциями:

- FunctionPointIndexOutOfBoundsException наследуется от IndexOutOfBoundsException и используется для случаев, когда программа пытается обратиться к точке функции по несуществующему индексу. Это непроверяемое исключение, так как ошибки индексации обычно являются ошибками программиста и должны быть исправлены в коде.
- InappropriateFunctionPointException наследуется от Exception и является проверяемым исключением. Оно используется при попытке выполнить операцию с точкой функции, которая нарушает целостность данных (например, добавление точки с x-координатой, которая нарушает упорядоченность точек). Это проверяемое исключение, так как такие ситуации могут возникать в нормальной работе программы и должны быть обработаны.

Задание 3

Были реализованы исключения для классов и методов:

- Два конструктора TabulatedFunction выбрасывают исключение IllegalArgumentException, если левая граница больше правой и если количество точек меньше двух;

```
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount){
    if (leftX >= rightX) {
        throw new IllegalArgumentException(s: "Левая граница области определения должна быть меньше правой");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть >=2");
    }
```

```
public ArrayTabulatedFunction(double leftX, double rightX, double[] values){

    if (leftX >= rightX) {
        throw new IllegalArgumentException(s: "Левая граница области определения должна быть меньше правой");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть >=2");
    }
```

- Методы getPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY() и deletePoint() выбрасывают исключение FunctionPointIndexOutOfBoundsException, если переданный индекс не входит в диапазон количества точек табулированной функции

```
public FunctionPoint getPoint(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
```

```
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException{
    if (point == null) {
        throw new IllegalArgumentException(s: "Точка без значения");
    }
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    if (index > 0 && doubleLessOrEquals(point.getX(), points[index - 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть больше предыдущей по x");
    }
    if (index < pointsCount - 1 &&  doubleGreaterOrEquals(point.getX(), points[index + 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть меньше следующей по x");
    }
```

```
public double getPointX(int index){
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
```

```
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    if (index > 0 && doubleLessOrEquals(x, points[index - 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая x-координата должна быть больше предыдущей");
    }
    if (index < pointsCount - 1 && doubleGreaterOrEquals(x, points[index + 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая x-координата должна быть меньше следующей");
    }
```

```
public double getPointY(int index){
    if (index <0 || index >= pointsCount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
```

```
public void setPointY(int index, double y) {
    if (index < 0 || index >= pointsCount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
```

```
public void deletePoint(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException(s: "Минимальное количество точек: 2");
    }
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
```

- Методы setPoint() и setPointX() выбрасывают исключение InappropriateFunctionPointException, если координата x точки не входит в диапазон значений, определяемый двумя соседними по индексу точками

```
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    if (index > 0 && doubleLessOrEquals(x, points[index - 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая x-координата должна быть больше предыдущей");
    }
    if (index < pointsCount - 1 && doubleGreaterOrEquals(x, points[index + 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая x-координата должна быть меньше следующей");
    }
```

```
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException{
    if (point == null) {
        throw new IllegalArgumentException(s: "Точка без значения");
    }
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    if (index > 0 && doubleLessOrEquals(point.getX(), points[index - 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть больше предыдущей по x");
    }
    if (index < pointsCount - 1 &&  doubleGreaterOrEquals(point.getX(), points[index + 1].getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть меньше следующей по x");
    }
```

- Метод addPoint выбрасывает исключение InappropriateFunctionPointException, если x координата предлагаемой точки совпадает с уже существующей.

```
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    if (point == null) {
        throw new IllegalArgumentException(s: "Точка не может быть null");
    }
    FunctionPoint newPoint = new FunctionPoint(point);
    int newIndex = 0;

    while (newIndex < pointsCount && doubleLess(points[newIndex].getX(), newPoint.getX())) {
     newIndex++;
    }
    if  (newIndex < pointsCount && doubleEquals(points[newIndex].getX(), newPoint.getX())) {
        throw new InappropriateFunctionPointException(message: "Такая точка уже существует");
    }
```

- Метод deletePoint выбрасывает исключение llegalStateException, если общее количество точек до удаления менее трех

```
public void deletePoint(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException(s: "Минимальное количество точек: 3");
    }
```

Задание 4

Была реализована альтернативная реализация табулированной функции на основе двусвязного циклического списка с выделенной головой. Двусвязный список в Java — это структура данных, в которой каждый узел содержит три поля: Данные (в нашем случае - точка функции FunctionPoint), ссылку на предыдущий элемент и ссылку на следующий элемент. Циклический список означает, что следующий указатель последнего узла указывает на первый, а предыдущий первого — на последний. Голова (head) - это специальный узел, который не содержит полезных данных (point = null), всегда существует и служит маркером начала и конца списка.

```
package functions;


public class LinkedListTabulatedFunction implements TabulatedFunction {
    private class FunctionNode {
        private FunctionPoint point;
        private FunctionNode previous;
        private FunctionNode next;

        public FunctionNode(FunctionPoint point){
            this.point = point;
            this.previous = null;
            this.next = null;
        }

        public FunctionNode(FunctionPoint point, FunctionNode previous, FunctionNode next){
            this.point = point;
            this.previous = previous;
            this.next = next;
        }

        public FunctionPoint getPoint() {
            return point;
        }

        public void setPoint(FunctionPoint point) {
            this.point = point;
        }

        public FunctionNode getPrevious() {
            return previous;
        }

        public FunctionNode getNext() {
            return next;
        }

        public void setPrevious(FunctionNode previous) {
            this.previous = previous;
        }

        public void setNext(FunctionNode next) {
            this.next = next;
        }
    }
```

Внутренний класс FunctionNode представляет элемент списка и содержит:

- Поле point для хранения данных (объект FunctionPoint)
- Поля previoius и next для ссылок на предыдущий и следующий элементы (являются объектами FunctionNode, так как ссылаются на другие узлы)
- Конструкторы для создания узлов: первый - без ссылок: для создания первого узла списка или временного узла; второй - с ссылками: для остальных целей
- Геттеры и сеттеры для доступа к полям

  Класс FunctionNode сделан внутренним и приватным, так как он используется только внутри LinkedListTabulatedFunction и не должен быть доступен извне.

```
private FunctionNode head;
private int pointsCount;
private FunctionNode lastAccessNode;
private int lastAccessIndex;
```

Основной класс LinkedListTabulatedFunction содержит:

- Поле head - голова списка
- Поле pointsCount - количество точек
- Поля lastAccessNode и lastAccessIndex - для оптимизации доступа

Все поля класса объявлены как private, а доступ к ним осуществляется через публичные методы-геттеры и сеттеры

- Методы:

1. Метод getNodeByIndex реализует оптимизированный доступ к элементам списка. Если запрашивается тот же элемент, что и в прошлый раз, или соседний элемент, доступ осуществляется за постоянное время O(1). В противном случае используется поиск с ближайшего конца (с начала для первых половины элементов, с конца для второй половины).
2. Метод addNodeToTail добавляет новый узел в конец списка, обновляя ссылки соседних узлов.
3. Метод addNodeByIndex вставляет новый узел в указанную позицию.
4. Метод deleteNodeByIndex удаляет узел по индексу, восстанавливая связи между предыдущим и следующим узлами.

Задание 5

```
public class LinkedListTabulatedFunction implements TabulatedFunction {

    //методы TabulatedFunction
    @Override
    public double getLeftDomainBorder(){
        if (pointsCount == 0) {
            throw new IllegalStateException(s: "Функция не содержит точек");
        }
        return head.getNext().getPoint().getX();
    }

    @Override
    public double getRightDomainBorder() {
        if (pointsCount == 0) {
            throw new IllegalStateException(s: "Функция не содержит точек");
        }
        return head.getPrevious().getPoint().getX();
    }

    @Override
    public double getFunctionValue(double x){
        if (doubleLess(x, getLeftDomainBorder()) || doubleGreater(x, getRightDomainBorder()) || pointsCount == 0) {
            return Double.NaN;
        }

        FunctionNode current;
        if (lastAccessNode != head && doubleGreaterOrEquals(x, lastAccessNode.getPoint().getX())) {
            current = lastAccessNode;
        } else {
            current = head.getNext();
        }

        while (current != head && current.getNext() != head) {
            double x_1 = current.getPoint().getX();
            double x_2 = current.getNext().getPoint().getX();

            if (doubleEquals(x, x_1)){
                lastAccessNode = current;
                lastAccessIndex = getNodeIndex(current);
                return current.getPoint().getY();
            }

            if (doubleEquals(x, x_2)){
                lastAccessNode = current.getNext();
                lastAccessIndex = getNodeIndex(current.getNext());
                return current.getNext().getPoint().getY();
            }

            if (doubleGreater(x, x_1) && doubleLess(x, x_2)){
                double y_1 = current.getPoint().getY();
                double y_2 = current.getNext().getPoint().getY();

                return (x - x_1) * (y_2 - y_1) / (x_2 - x_1) + y_1;
            }
```

```
public double getFunctionValue(double x){
        current = current.getNext();
    }
    return Double.NaN;
}

@Override
public int getPointsCount(){
    return pointsCount;
}

@Override
public FunctionPoint getPoint(int index){
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    FunctionNode node = getNodeByIndex(index);
    return new FunctionPoint(node.getPoint());
}

@Override
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException{
    if (point == null) {
        throw new IllegalArgumentException(s: "Точка без значения");
    }

    FunctionNode node = getNodeByIndex(index);

    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    if (index > 0 && doubleLessOrEquals(point.getX(), node.getPrevious().getPoint().getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть больше предыдущей по x");
    }
    if (index < pointsCount - 1 && doubleGreaterOrEquals(point.getX(),node.getNext().getPoint().getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть меньше следующей по x");
    }

    node.setPoint(new FunctionPoint(point));
}

@Override
public double getPointX(int index){
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    return getNodeByIndex(index).getPoint().getX();
}
```

```java
@Override
public void setPointX(int index, double x) throws InappropriateFunctionPointException{
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }

    FunctionNode node = getNodeByIndex(index);
    FunctionPoint currentPoint = node.getPoint();

    if (index > 0 && doubleLessOrEquals(x, node.getPrevious().getPoint().getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть больше предыдущей по x");
    }
    if (index < pointsCount - 1 && doubleGreaterOrEquals(x, node.getNext().getPoint().getX())) {
        throw new InappropriateFunctionPointException(message: "Новая точка должна быть меньше следующей по x");
    }

    node.setPoint(new FunctionPoint(x, currentPoint.getY()));
}

@Override
public double getPointY(int index){
    if (index <0 || index >= pointsCount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }
    return getNodeByIndex(index).getPoint().getY();
}

@Override
public void setPointY(int index, double y) {
    if (index < 0 || index >= pointsCount){
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }

    FunctionNode node = getNodeByIndex(index);
    FunctionPoint currentPoint = node.getPoint();
    node.setPoint(new FunctionPoint(currentPoint.getX(), y));
}

@Override
public void deletePoint(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException(s: "Минимальное количество точек: 2");
    }
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " не входит в диапазон");
    }

    deleteNodeByIndex(index);
}
```

```java
@Override
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    if (point == null) {
        throw new IllegalArgumentException(s: "Точка без значения");
    }

    FunctionNode current = head.getNext();
    int newIndex = 0;

    while (current != head && doubleLess(current.getPoint().getX(), point.getX())) {
        current = current.getNext();
        newIndex++;
    }
    if  (newIndex < pointsCount && doubleEquals(current.getPoint().getX(), point.getX())) {
        throw new InappropriateFunctionPointException(message: "Такая точка уже существует");

    }
    FunctionNode newNode = addNodeByIndex(newIndex);
    newNode.setPoint(new FunctionPoint(point));
}

private int getNodeIndex(FunctionNode node) {
    FunctionNode current = head.getNext();
    int index = 0;
    while (current != head) {
        if (current == node){
            return index;
        }
        current = current.getNext();
        index++;
    }
    throw new IllegalStateException (s: "Узел не найден в списке");
}

private boolean doubleEquals(double a, double b) {
    return Math.abs(a - b) < EPSILON;
}

private boolean doubleLess(double a, double b) {
    return a < b - EPSILON;
}

private boolean doubleGreater(double a, double b) {
    return a > b + EPSILON;
}
 private boolean doubleLessOrEquals(double a, double b) {
    return a < b + EPSILON;
}

private boolean doubleGreaterOrEquals(double a, double b) {
    return a > b - EPSILON;
}
```

В классе LinkedListTabulatedFunction реализованы все методы, аналогичные классу на массиве, но с использованием возможностей связного списка.

Конструкторы создают функцию тем же способом, что и ArrayTabulatedFunction, но используют внутренние методы работы со списком для добавления точек.

Метод getFunctionValue оптимизирован за счет того, что поиск интервала для интерполяции начинается с последнего доступного узла, что ускоряет работу при последовательном доступе к близким значениям x.

Методы setPoint и setPointX используют прямые ссылки на соседние узлы для проверки упорядоченности, что более эффективно, чем доступ по индексу в массиве.

Метод addPoint осуществляет прямой обход списка для поиска позиции вставки, что естественно для связной структуры.

Метод getNodeIndex(FunctionNode node) (вспомогательный) является ключевым для оптимизации, так как позволяет обновлять кэш lastAccessIndex при прямом доступе к узлам

Задание 6

```java
package functions;

public interface TabulatedFunction {

    double getLeftDomainBorder();

    double getRightDomainBorder();

    double getFunctionValue(double x);

    int getPointsCount();

    FunctionPoint getPoint(int index);

    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;

    double getPointX(int index);

    void setPointX(int index, double x) throws InappropriateFunctionPointException;

    double getPointY(int index);

    void setPointY(int index, double y);

    void deletePoint(int index);

    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;

}
```

Создан интерфейс TabulatedFunction, который определяет общее поведение для всех табулированных функций, независимо от их внутренней реализации. Интерфейс содержит объявления всех основных методов работы с функцией: получение границ области определения, вычисление значения функции, работу с точками и т.д.

Класс TabulatedFunction переименован в ArrayTabulatedFunction для лучшего отражения его сущности.

Оба класса (ArrayTabulatedFunction и LinkedListTabulatedFunction) теперь реализуют интерфейс TabulatedFunction (ArrayTabulatedFunction implements TabulatedFunction; public class LinkedListTabulatedFunction implements TabulatedFunction). Это позволяет работать с функциями через интерфейс, не заботясь о конкретной реализации.

Для корректности работы перед методами классов ArrayTabulatedFunction и LinkedListTabulatedFunction, описанными в интерфейсе необходимо писать @Override - это аннотация в Java, которая указывает компилятору, что метод, помеченный этой аннотацией, реализует метод интерфейса.

Задание 7

Main.java > Language Support for Java(TM) by Red Hat > Main > testingFunction(TabulatedFunction)

```java
import functions.*;

public class Main {
    Run main | Debug main
    public static void main(String[] args) {
        System.out.println(x: "_ArrayTabulatedFunction_");
        testingFunction(new ArrayTabulatedFunction(leftX: 0, rightX: 5, new double[]{0, 1, 8, 27, 64, 125}));

        System.out.println(x: "\n_LinkedListTabulatedFunction_");
        testingFunction(new LinkedListTabulatedFunction(leftX: 0, rightX: 5, new double[]{0, 1,8, 27, 64, 125}));

        System.out.println(x: "\n_Exception test_");
        testingExceptions();
    }

    public static void testingFunction(TabulatedFunction func) {
        System.out.println(x: "y = x^3, x = [0,5]");
        printFunctionInfo(func);

        System.out.println(x: "\ncalculating values");
        double[] testPoints = {-1, 0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 5};
        for (double x : testPoints) {
            double result = func.getFunctionValue(x);
            if (Double.isNaN(result)) {
                System.out.printf(format: "f(%.1f) = not defined%n", x);
            } else {
                System.out.printf(format: "f(%.1f) = %.2f%n", x, result);
            }
        }

        System.out.println(x: "\nadding points");
        try {
            func.addPoint(new FunctionPoint(x: 0.5, y: 0.125));
            func.addPoint(new FunctionPoint(x: 2.5, y: 15.625));
            System.out.println(x: "result:");
            printFunctionInfo(func);
        } catch (InappropriateFunctionPointException e) {
            System.out.println(e.getMessage());
        }

        System.out.println(x: "\ncheck");
        for (double x : new double[]{0.5, 2.5}) {
            System.out.printf(format: "f(%.1f) = %.3f%n", x, func.getFunctionValue(x));
        }

        System.out.println(x: "\npoint changing");
        func.setPointY(index: 2, y: 5.0);
        System.out.println(x: "result:");
        printFunctionInfo(func);
        System.out.printf(format: "f(%.1f) = %.3f%n", func.getPointX(index: 2), func.getFunctionValue(func.getPointX(index: 2)));

        System.out.println(x: "\npoint deleting");
        func.deletePoint(index: 3);
        System.out.println(x: "result:");
        printFunctionInfo(func);
```

```
    private static void printFunctionInfo(TabulatedFunction func) {
        System.out.println("Область определения: [" + func.getLeftDomainBorder() + ", " + func.getRightDomainBorder() + "]");
        System.out.println("Количество точек: " + func.getPointsCount());
        System.out.print(s: "Точки: ");
        for (int i = 0; i < func.getPointsCount(); i++) {
            System.out.print("(" + func.getPointX(i) + "; " + func.getPointY(i) + ")");
            if (i < func.getPointsCount() - 1) {
                System.out.print(s: ", ");
            }
        }
        System.out.println();
    }

    private static void testingExceptions() {
        System.out.println(x: "_invalid parameters test_");
        try {
            TabulatedFunction func = new ArrayTabulatedFunction(leftX: 5, rightX: 0, pointsCount: 5);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }

        try {
            TabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 5, pointsCount: 1);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }

        System.out.println(x: "_invalid index test_");
        TabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 5, new double[]{0, 1, 8, 27, 64, 125});
        try {
            func.getPointX(-1);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println(e.getMessage());
        }

        try {
            func.getPointY(index: 10);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println(e.getMessage());
        }

        System.out.println(x: "_invalid operations test_");
        try {
            func.setPoint(index: 1, new FunctionPoint(x: 0, y: 100));
        } catch (InappropriateFunctionPointException e) {
            System.out.println(e.getMessage());
        }

        try {
            func.addPoint(new FunctionPoint(x: 2, y: 100));
        } catch (InappropriateFunctionPointException e) {
            System.out.println(e.getMessage());
        }
```

```
        System.out.println(x: "_invalid operations test_");
        try {
            func.setPoint(index: 1, new FunctionPoint(x: 0, y: 100));
        } catch (InappropriateFunctionPointException e) {
            System.out.println(e.getMessage());
        }

        try {
            func.addPoint(new FunctionPoint(x: 2, y: 100));
        } catch (InappropriateFunctionPointException e) {
            System.out.println(e.getMessage());
        }

        System.out.println(x: "_invalid delete operation test_");
        TabulatedFunction deletedFunc = new ArrayTabulatedFunction(leftX: 0, rightX: 1, new double[]{0, 1});
        try {
            deletedFunc.deletePoint(index: 0);
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }

        System.out.println(x: "_point null test_");
        try {
            func.addPoint(point: null);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        } catch (InappropriateFunctionPointException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Реализован класс Main для проверки работы разработанных классов

Тестирование базовой функциональности включает:

- Создание функций обоих типов
- Вычисление значений в различных точках (внутри области определения, на границах, вне области определения)
- Добавление новых точек с сохранением упорядоченности
- Изменение существующих точек
- Удаление точек

Тестирование обработки исключений проверяет все сценарии, когда должны выбрасываться исключения:

- Некорректные параметры конструкторов
- Выход за границы индексов при доступе к точкам
- Нарушение упорядоченности при изменении точек
- Попытка добавить точку с существующей x-координатой
- Попытка удалить точку, когда останется меньше двух точек
- Передача null в методы

Консоль:

```
lisako@lisa MINGW64 ~/Documents/oop/Lab-3-2025 (main)
javac functions/*.java
javac main.java
java Main
```

ArrayTabulatedFunction

```
y = x^3, x = [0,5]
Область определения: [0.0, 5.0]
Количество точек: 6
Точки: (0.0; 0.0), (1.0; 1.0), (2.0; 8.0), (3.0; 27.0), (4.0; 64.0), (5.0; 125.0)
calculating values
f(-1,0) = not defined
f(0,0) = 0,00
f(0,5) = 0,50
f(1,0) = 1,00
f(1,5) = 4,50
f(2,0) = 8,00
f(2,5) = 17,50
f(3,0) = 27,00
f(3,5) = 45,50
f(4,0) = 64,00
f(5,0) = 125,00
adding points
result:
Область определения: [0.0, 5.0]
Количество точек: 8
Точки: (0.0; 0.0), (0.5; 0.125), (1.0; 1.0), (2.0; 8.0), (2.5; 15.625), (3.0; 27.0), (4.0;
64.0), (5.0; 125.0)
check
f(0,5) = 0,125
f(2,5) = 15,625
point changing
result:
Область определения: [0.0, 5.0]
Количество точек: 8
Точки: (0.0; 0.0), (0.5; 0.125), (1.0; 5.0), (2.0; 8.0), (2.5; 15.625), (3.0; 27.0), (4.0;
64.0), (5.0; 125.0)
f(1,0) = 5,000
point deleting
result:
Область определения: [0.0, 5.0]
Количество точек: 7
Точки: (0.0; 0.0), (0.5; 0.125), (1.0; 5.0), (2.5; 15.625), (3.0; 27.0), (4.0; 64.0), (5.0;
125.0)
```

LinkedListTabulatedFunction

```
y = x^3, x = [0,5]
Область определения: [0.0, 5.0]
Количество точек: 6
Точки: (0.0; 0.0), (1.0; 1.0), (2.0; 8.0), (3.0; 27.0), (4.0; 64.0), (5.0; 125.0)
calculating values
f(-1,0) = not defined
f(0,0) = 0,00
f(0,5) = 0,50
f(1,0) = 1,00
f(1,5) = 4,50
f(2,0) = 8,00
f(2,5) = 17,50
f(3,0) = 27,00
f(3,5) = 45,50
f(4,0) = 64,00
f(5,0) = 125,00
adding points
result:
Область определения: [0.0, 5.0]
Количество точек: 8
Точки: (0.0; 0.0), (0.5; 0.125), (1.0; 1.0), (2.0; 8.0), (2.5; 15.625), (3.0; 27.0), (4.0;
64.0), (5.0; 125.0)
check
f(0,5) = 0,125
f(2,5) = 15,625
point changing
result:
Область определения: [0.0, 5.0]
Количество точек: 8
```

```
Точки: (0.0; 0.0), (0.5; 0.125), (1.0; 5.0), (2.0; 8.0), (2.5; 15.625), (3.0; 27.0), (4.0;
64.0), (5.0; 125.0)
f(1,0) = 5,000
point deleting
result:
Область определения: [0.0, 5.0]
Количество точек: 7
Точки: (0.0; 0.0), (0.5; 0.125), (1.0; 5.0), (2.5; 15.625), (3.0; 27.0), (4.0; 64.0), (5.0;
125.0)
```

Exception test

```
_invalid parameters test_
Левая граница области определения должна быть меньше правой
Количество точек должно быть >=2
_invalid index test_
Индекс -1 не входит в диапазон
Индекс 10 не входит в диапазон
_invalid operations test_
Новая точка должна быть больше предыдущей по x
Такая точка уже существует
_invalid delete operation test_
Минимальное количество точек: 3
_point null test_
Точка не может быть null
```